Lesson 4
File I/O and Data Manipulation

OUTLINE

FILE I/0

- Opening a file
- Reading data from the file
- Writing data to the file
- Closing a file
- Creating and deleting a file
- Other os Module methods

PYTHON FILE I/O

Files

Files are named locations on disk to store related information. They are used to permanently store data in a non-volatile memory (e.g. hard disk).

When we want to read from or write to a file, we need to open it first. When we are done, it needs to be closed so that the resources that are tied with the file are freed.

Hence, in Python, a file operation takes place in the following order:

PYTHON FILE I/O

Opening a file

Python has a built-in open() function to open a file. This function returns a file object, also called a handle, as it is used to read or modify the file accordingly.

Mode	Description
r	Opens a file for reading (default)
w	Opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists
x	Opens a file for exclusive creation. If the file already exists, the operation fails
a	Opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist
t	Opens in text mode (default)
b	Opens in binary mode
+	Opens a file for updating (reading and writing)

```
s = open("willi.txt")

s = open("willi.txt",'w')  # write in text mode
s = open("myImg.bmp",'r+b') # read and write in binary mode
```

```
s= open("willi.txt", mode='r', encoding='utf-8')
```

When working with files in text mode, it is highly recommended to specify the encoding type.

Closing Files in Python

When we are done with performing operations on the file, we need to properly close the file. Closing a file will free up the resources that were tied with the file. It is done using the close() method available in Python.

```
s = open("test.txt", encoding = 'utf-8')
s.close
```

This method is not entirely safe. If an exception occurs when we are performing some operation with the file, the code exits without closing the file

```
try:
  s = open("test.txt", encoding = 'utf-8')
finally:
  f.close()
```

A safer way close a file

```
with open("test.txt", encoding = 'utf-8')
as f:
```

The best way to close a file is by using the with statement. This ensures that the file is closed when the block inside the with statement is exited

We don't need to explicitly call the close() method. It is done internally

PYTHON FILE I/O

Writing to Files in Python

In order to write into a file in Python, we need to open it in write w, append a or exclusive creation x mode.

```
with open("Myfile.txt",'w',encoding = 'utf-8') as myFile:
 myFile.write("Willis College Header\n")
 myFile.write(" Willis College body1\n\n")
 myFile.write(" Willis College body12\n")
```

Reading Files in Python

To read a file in Python, we must open the file in reading r mode.

There are various methods available for this purpose. We can use the read(size) method to read in the size number of data. If the size parameter is not specified, it reads and returns up to the end of the file.

```
myFile = open("test.txt",'r',encoding = 'utf-8')
myFile.read(10)    # read the first 10 data
myFile.tell()    # get the current file position
myFile.readline()#read individual lines of a file.
myFilereadlines() # read list of remaining lines of the entire file
```

```
for line in myFile:
    print(line, end = '')
```

PYTHON FILE I/O

File Methods

Method	Description
close()	Closes an opened file. It has no effect if the file is already closed
detach()	Separates the underlying binary buffer from the TextIOBase and returns it
fileno()	Returns an integer number (file descriptor) of the file
flush()	Flushes the write buffer of the file stream
isatty()	Returns True if the file stream is interactive
read(n)	Reads at most n characters from the file Reads until end of file if it is negative or None
readable()	Returns True if the file stream can be read from

PYTHON FILE I/O

File Methods

Method	Description
readline(n=-1)	Reads and returns one line from the file. Reads in at most n bytes if specified.
readlines(n=-1)	Reads and returns a list of lines from the file. Reads in at most n bytes/characters if specified.
seek(offset,from=SEEK_SET)	Changes the file position to offset bytes, in reference to from (start, current, end).
seekable()	Returns True if the file stream supports random access.
tell()	Returns an integer that represents the current position of the file's object.
truncate(size=None)	Resizes the file stream to size bytes. If size is not specified, resizes to current location.
writable()	Returns True if the file stream can be written to.
write(s)	Writes the string s to the file and returns the number of characters written.
writelines(lines)	Writes a list of lines to the file.

DELETING A FILE

We use the Remove() method to delete files by supplying the name of the file to be deleted as the argument.

Syntax
os.remove(file_name)

Examples

```
import os

#  Delete colours.txt file
os.remove("colours.txt")
```

OTHER OS MODULE METHODS

The OS module in Python provides functions for interacting with the operating system. OS comes under Python's standard utility modules

OS Module	Description
os.getcwd()	To get the location of the current working directory.The folder where the Python script is running is known as the Current Directory.
os.chdir()	To change the current working directory
os.mkdir()	To create a directory named path with the specified numeric mode. This method raises FileExistsError if the directory to be created already exists.
os.makedirs()	To create a directory recursively.os.makedirs() method will create all missing intermediate-level directory
os.listdir()	method in Python is used to get the list of all files and directories in the specified directory.If we don't specify any directory, then the list of files and directories in the current working directory will be returned.
os.remove()	To remove or delete a file path. This method can not remove or delete a directory. If the specified path is a directory then OSError will be raised by the method.
os.rmdir()	os.rmdir() method in Python is used to remove or delete an empty directory. OSError will be raised if the specified path is not an empty directory.
os.rename()	To rename a file
os.path.exists()	To check whether a file exists or not by passing the name of the file as a parameter.
os.path.getsize()	To get the size of the file in bytes

CLASS DEMOSTRATION

Class Demonstration 1: Write a Python script that write the below list to a called cars.txt file.

car = ['Toyota', 'Benz', 'Kia', 'Honda', 'Ferrarri', 'Lexus']

Class Demonstration 2: Write a Python script to get the file size of a pythonclose.txt file.

Class Demonstration 3: Write a python script that copies cars.txt file to another file cars_two.txt.

Class Demonstration 4: Write a Python program to read a file line by line and store it into a variable.

COMPLETE THE FOLLOWING QUESTION

Write a Python function that accept the file name as parameter and read the file line by line and store it into a list.

1. Using the cars.txt, call your function and pass the file to your function
2. Print the created list to your console.

COMPLETE THE FOLLOWING QUESTION

Write a function that accepts the file name as argument, read from car.txt and display all words that are less than 5 characters.

Modules and Packages

OUTLINE

MODULES

- Introduction
- Overview
- Importing a module
- Executing a module as a script
- Reloading a module
- Python Module Search Path
- Exploring modules

PACKAGES

- Package initialization
- Importing from a package
- Sub packages

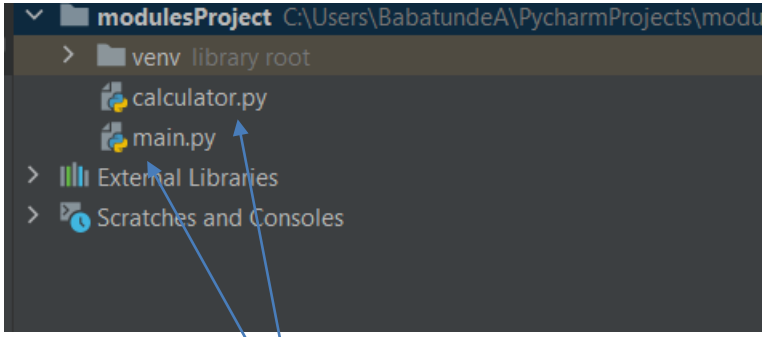
MODULES

In programming, a module is a piece of software that has a specific functionality. Example a module: A calculator module that computes addition, subtraction, division and multiplication of two numbers.

Writing modules

Modules in Python are just Python files with a .py extension. The name of the module is the same as the file name. A Python module can have a set of functions, classes, or variables defined and implemented

For example, calculator.py is a module and the module name would be calculator.



Modules

NB: The first time a module is loaded into a running Python script, it is initialized by executing the code in the module once. If another module in your code imports the same module again, it will not be loaded again

MODULES – PEP 8.0 MODULE CODING STANDARD

- Module Names:
  - Short, lowercase names, without underscores.
  - Example: data.py

- Organization Import
  - They should be always put at the top of the file, just after any module comments and docstrings, and before module global and constants.
  - Imports should be on separate lines.

    Wrong: import sys, os

    Correct: import sys
    import os
  - Imports should be grouped in the following order with a blank line between each group of imports:
    - standard library imports
    - related major package imports
    - application specific imports

MODULES

calculate.py

```
def add(num_one, num_two):
    """
    This function adds two numbers
    :param num_one:
    :param num_two:
    :return:
    """
    return num_one + num_two


def multiply(num_one, num_two):
    """
    This function multiply two numbers
    :param num_one:
    :param num_two:
    :return:
    """
    return num_one * num_two


def subtract(num_one, num_two):
    """
    This function subtracts two numbers
    :param num_one:
    :param num_two:
    :return:
    """
    return num_one + num_two


def divide(num_one, num_two):
    """
    This function divides number+one by number two
    :param num_one:
    :param num_two:
    :return:
    """
    return num_one / num_two
```

main.py

```
import calculator as cal

# this means that if this script is
executed, then
# main() will be executed
if __name___== '__main__':
    print(cal.add(1, 5))
    print(cal.divide(1, 5))
    print(cal.multiply(1, 5))
    print(cal.subtract(6, 9))
```

The Python script main.py implements the calculator module. It uses the functions from the file calculate.py.

NB: Modules are imported from other modules using the import command.

MODULES – CALCULATOR.PY AND MAIN.PY EXPLAINED

In this example, the main module imports the calculate module, which enables it to use functions implemented in that module. To use the function, add, subtract, divide and multiple function from the calculate module, we need to specify in which module the function is implemented, using the dot operator.

To reference the add function from the calculate module, we need to import the calculate module and then call calculate.add(value_one.value_two) or using cal as the alias in case, cal.add(value_one.value_two)

When the import calculate directive runs, the Python interpreter looks for a file in the directory in which the script was executed with the module name and a .py suffix. In this case it will look for calculate.py. If it is found, it will be imported. If it's not found, it will continue looking for built-in modules.

MODULES - IMPORTING MODULE OBJECTS

Importing module objects to the current namespace

A namespace is a system where every object is named and can be accessed in Python. We can import specific names from a module without importing the module as a whole.

Example
We import the function add into the main script's namespace by using the from command.

Syntax
from {methodname} import {functionName}

```
from calculator import add
Print(add(1,2))
```

You will notice we are not making any reference to the module name when we called the add function

MODULES - IMPORTING ALL OBJECTS

Importing all objects to the current namespace

We can use the import * command to import all the objects in a module

Syntax
from {methodname} import *

```
from calculator import *
Print(add(1,2))
```

This will import all the functions (add, subtract, multiple and divide)

```
from math import *
Print("The value of Pi is", pi)
```

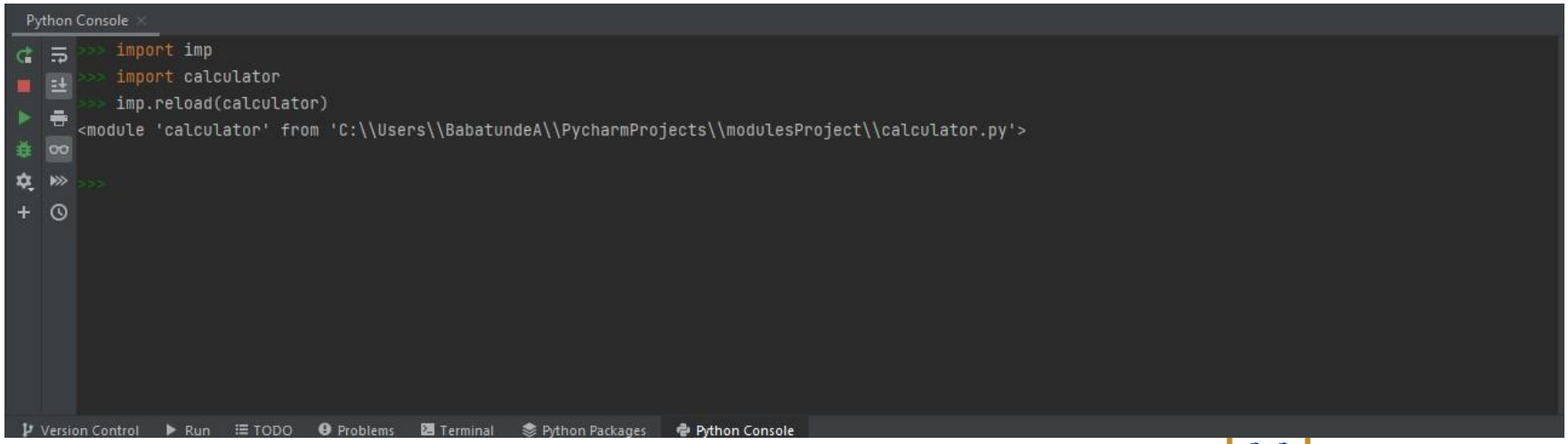
MODULES - RELOADING A MODULE

Reloading a Module

The Python interpreter imports a module only once during a session. This makes things more efficient.

To reload a module, We can use the reload() function inside the imp module to reload a module.

```
import imp
Import {moduleName}
imp.reload({moduleName})
```



MODULES - PYTHON MODULE SEARCH PATH

Python Module Search Path

Python looks at several places when importing a module. Interpreter first looks for a built-in module. Then(if built-in module not found), Python looks into a list of directories defined in sys.path.

Type sys.path in your PyCharm Console

```
['C:\\Program Files\\JetBrains\\PyCharm Community Edition
2021.3.3\\plugins\\python-ce\\helpers\\pydev',
'C:\\Program Files\\JetBrains\\PyCharm Community Edition
2021.3.3\\plugins\\python-ce\\helpers\\third_party\\thriftpy',
'C:\\Program Files\\JetBrains\\PyCharm Community Edition
2021.3.3\\plugins\\python-ce\\helpers\\pydev'
]
```

NB: We can add more paths, delete, remove or edit a particular path. We use some of the list method operations to perform this.

Some list Operators:

append, pop, clear, insert, copy, extend,

Exploring modules

The two most important functions that comes in handy when exploring modules in Python are:

- The dir function
  This shows the functions that are implemented in reach module

```
Type the following on your python console
Import math
dir(math)
math.___doc___
```

NB: The respective module needs to be imported before get more information about them. Run this in your python console

- The help function
  We are read more about the function in the module we want to use using the help function.

```
Type the following on your python console

Import math
help(math)
```

COMPLETE THE FOLLOWING QUESTION

1. Using the dir function, get all the function in the calculator module. Create a modulus function on the calculator module and then implement the function on main module.
2. Using the help function, get more details regarding the calculator module

COMPLETE THE FOLLOWING QUESTION

Create module call it "productcal.py" that has the two functions.

- Calculate the discount of the product. The function should accept the product price, and quantity as arguments and return the discount
  - Discount should follow the below rules
    - If quantity >0 and quantity <=10, discount = 5%
    - If quantity >10 and quantity <=50. discount = 10%
    - If quantity >50 and quantity <=100. discount = 15%
    - If quantity >100. discount = 20%
  - *discount* = {*discount percentage*}/100*{*productAmount*}*{*quantity*}

- Calculate the tax of the product where the product price, discount, quantity are passed as a parameter. The function should accept the product price as argument and return the tax.
  - Tax = (1.5 * (({*productAmount*}*{*quantity*}) - {*discount*}))/100

Create another module and call it "producttest".

- Import the productcal
- Write a python statement that reads the quantity, product price from the user and calculate the amount payable.
- Amount payable = (({productAmount}*{quantity))-{Discount}) + Tax

Create two different functions that checks the validity of quantity and price

NB:

- Ensure you check that the quality is >0
- Ensure you check you only accept integer for quantity
- Ensure you check you only accept integer or float for price
- Keep asking the user to enter a value input until quantity is >0 and quantity is an integer. Also keep asking the user for price input until the input is either integer or float. Write two different functions that checks quantity and price

Test your code with the following values

- Quantity=50, price=1000
- Quantity=50, price=56.90
- Quantity=aaaaa, price=1000
- Quantity=0, price=1000

PACKAGES

As our application program grows larger in size with a lot of modules, we can place similar modules in one package and different modules in different packages. This makes a project (program) easy to manage and conceptually clear.

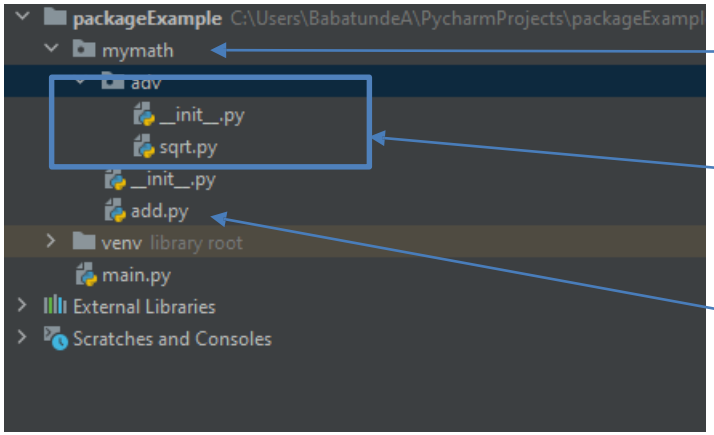
Packages are namespaces containing nested packages (sub packages) and modules.

Each package in Python is a directory which **MUST** contain a file called ___init___.py in order for Python to consider it as a package. This file can be left empty, but we generally place the initialization code for that package in this file.

The ___init___.py file can also decide which modules the package exports as the API, while keeping other modules internal

```
___init___.py:
___all____= [“{functionname}"]
```

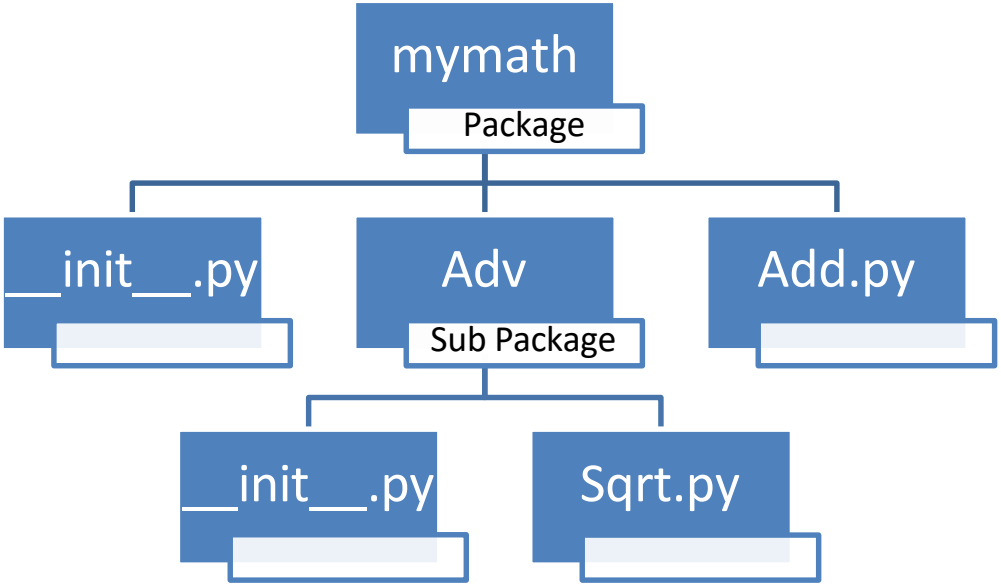
PACKAGES



Main Package

Sub Package

Module



PACKAGES –INSTALLING PACKAGES

There are two major ways to install packages to your python project

1. Go the python packages tab then search the required package you need and install (For more information, go to lesson 2)
2. You can install package from the terminal
    1. pip install *{packagename}*

    Example,
    Pip install encode - This will download and install the encode package.

NB: To see the installed package, go the Venv folder and then Lib folder and then Site-packages or checked install section in Python package window

Package Naming convention

Use a short, lowercase word or words. Do not separate words with underscores.

Example
package, mypackage

PACKAGES -IMPORTING MODULE FROM PACKAGE

Importing module from package

We can import modules from packages using the dot (.) operator.

Syntax
Import {*packagename*}.{modulename}

Or

from {*packagename*} import {modulename}

Syntax
Import {*packagename*}. {*subpackagename*}.{modulename}

Or

from {*packagename*}. {*subpackagename*} import {modulename}

When we have sub packages

COMPLETE THE FOLLOWING QUESTION

1. Create a new python project and call it modpackageproject
2. Under modpackageproject, Create a package and call it myutility
3. In the myutility package, create a python file and called rgenerator.py
    1. Write a function in rgenerator.py that returns a random numbers between 0 and 1
4. Create a sub package under myutility package and called it adv
    1. Under adv, create a python file and called advgenetor.py
        1. Write a function in advgenetor that returns a random numbers between 0 and 1

5. Under modpackageproject, create another package and call it imp
    1. Under imp, create a new python script and call it main
        1. import and implement rgenerator and advgenetor modules via the packages
        2. Print the random numbers generated for rgenerator and advgenetor module.

COMPLETE THE FOLLOWING QUESTION

1. Create a new python project and call it advmodpackageproject
2. Under advmodpackageproject, Create a package and call it utility
3. In the utility package,
    1. Create a python file and called median.py. Write a function in median.py that returns a median of 5 numbers that has been passed as argument.
    2. Create another python file and called average.py. Write a function in average.py that returns a average of 5 numbers that has been passed as argument.
    3. Create a sub package under utility package and called it adv
    4. Under adv, create a python file and called pi.py
        1. Write a function in pi that returns a PI value(3.1415...)

        Use the Math function to generate pi
4. Under advmodpackageproject, create another package and call it imp
    1. Under imp, create a new python script and call it main
        1. import and implement median and pi modules via the packages
        2. Print the random numbers generated for median and pi module.